



Tecnológico de Monterrey

Challenge-Submission 3

Modeling of Multi-Agent Systems with Computer Graphics (Gpo 102)

Fernando Gael Hernández Salazar — A01029264

Marco Antonio Torres Ramírez — A00839451

Gustavo Axel Huerta García — A00574308

Alfredo Luce Morales — A01772499

Cedric Schuster — A01831137

Campus Monterrey

María Raquel Landa Cavazos | Luis Alberto Muñoz Ubando

September 7th, 2025

1. Introduction.....	3
2. Team identity.....	3
2.1 Credits (members).....	4
3. Context and Problem.....	4
4. General Objectives.....	5
5. Restrictions.....	6
6. Requirements.....	6
7. Description of the multiagent system.....	7
7.1 Agent model.....	7
7.2 Environmental model.....	8
7.3 Negotiation model.....	8
7.4 Interaction model.....	10
7.5 Deep Q-Learning.....	15
7.6 Generation of JSON.....	15
7.7 Connection: server/API.....	16
7.8 Unity client integration.....	17
8. Description of the graphic modeling.....	17
8.1 Scene to model.....	17
8.2 Graphic components.....	18
8.3 Prefabs.....	22
8.4 Scripts.....	23
9. Project management deliverables.....	27
10. References.....	27

1. Introduction

As urban populations grow, the challenge of managing municipal solid waste has reached a critical scale. In 2020, the world generated 2.24 billion tonnes of waste, a figure projected to surge to 3.88 billion tonnes by 2050 (Gautam & Velijala, 2024). Cities are the epicenter of this issue, with daily waste generation per person steadily increasing. This project addresses the urgent need for innovative solutions by designing a multi-agent system to optimize the mobility and efficiency of autonomous trash collection units. Our proposed solution is a 3D simulation, built in Unity, that models how multiple agents can intelligently navigate a complex urban environment, coordinate to collect different types of waste from various points, and adapt to real-world obstacles and constraints, thereby providing a scalable and efficient alternative to traditional, static collection methods.

2. Team identity

Team 4

Name: Grid & Grit

Grid reflects structure, connectivity, and efficiency, representing the urban layout (streets, transport routes, smart logistics).

Grit embodies resilience, determination, and adaptability, highlighting the team's commitment to solving urban challenges with persistence and creativity.

Together, the name conveys a balance between technical precision and human perseverance.

Slogan: *"Smart routes. Cleaner streets"*

Smart routes emphasizes the use of technology, optimization, and data-driven solutions to design better transportation and logistics networks.

Cleaner streets reflect the environmental and social impact, aiming at reducing pollution, congestion, and inefficiencies.

The slogan is short, memorable, and connects both the innovation and sustainability aspects of the project.

Logo:



The grid of squares represents networks, city blocks, and connectivity, symbolizing how routes and logistics interconnect.

The checkmark illustrates efficiency, correctness, and achieved goals, reinforcing the idea of optimization and reliability.

Minimalist and geometric, which conveys clarity, modernity, and professionalism.

Easy to recognize and scalable for different applications (presentations, documents, branding).

Color: **green**

Green symbolizes sustainability, growth, and eco-friendly practices, perfectly aligned with the slogan “Cleaner streets.”

It also provides a sense of trust, harmony, and innovation, making it appealing in contexts related to technology and the environment.

Team Values

1. Innovation: Using smart technology for logistics and routing.
2. Sustainability: Prioritizing eco-friendly and socially responsible solutions.
3. Collaboration: Working as a connected “grid” where every member contributes.
4. Resilience: Tackling challenges with grit and determination.

Grid & Grit is a team inspired by the principles of Multi-Agent Systems. Like agents on a grid, each member contributes autonomously yet cooperatively, ensuring efficient problem-solving and resilient solutions. Guided by the balance of structure (Grid) and perseverance (Grit), the team develops smart routing strategies that promote cleaner, more sustainable urban environments. Through collaboration, adaptability, and optimization, Grid & Grit embodies the spirit of distributed intelligence applied to real-world challenges.

2.1 Credits (members)

Name	Position
Fernando Gael Hernández Salazar	Lead Programmer & AzureDevOps Engineer
Marco Antonio Torres Ramírez	Multi-Agent System Designer
Gustavo Axel Huerta García	3D Graphics & Unity Specialist
Alfredo Luce Morales	UI/UX & Dashboard Designer
Cedric Schuster	Project Manager & Documentation

3. Context and Problem

The specific problem we are addressing is the logistical inefficiency of urban waste collection in the face of increasing waste generation and complex city spaces. Traditional collection routes are often rigid, falling to adapt to dynamic factors like traffic congestion, road blockages, and variable fill levels of trash containers. This leads to wasted resources, increasing operational costs, and environmental issues like overflowing bins (Ferrer & Alba, 2018). The challenge is to design a system

where collection units can perceive their environment, make intelligent decisions, and cooperate to achieve a common goal.

Recent research highlights that adaptive strategies are essential. For example, Bharadhwak et al. (2025) showed that dynamic routing systems integrating IoT sensors, real time traffic, and dashboards can achieve up to a 22% reduction in collection time compared to static methods. Similarly, Studies by Karadimas et al. (n. d) and Likotiko et al. (2017) demonstrate how multi agent based simulations integrated with GIS to capture the spatial and temporal complexity of waste collection can enable agents to perceive their environment, adapt routes, and coordinate with each other intelligently. Meanwhile, Wouda et al. (2024) demonstrated optimized routes and vehicle allocation, reducing costs by more than 40% while maintaining service reliability.

Our mission is to design and simulate a multi-agent solution that optimizes the mobility of autonomous trash collection units in a realistic urban setting. The goal is to create a 3D simulation that demonstrates how multiple agents can collaboratively navigate the environment, collect waste, and return to a central depot efficiently, all while overcoming real-world challenges like obstacles and dynamic task allocation. The system will also feature a system evolution dashboard to monitor key performance indicators (KPIs) in real-time, allowing for iterative improvement and analysis of the system's performance under various conditions.

4. General Objectives

Objective 1: To improve the efficiency of urban waste collection by reducing the operational inefficiencies caused by rigid routes, static schedules, and lack of adaptability in traditional systems.

Objective 2: To evaluate how the performance of autonomous collection units can be optimized through real-time monitoring and adaptive decision-making, focusing on metrics such as collection time, coverage, and resource utilization.

Objective 3: To simulate a realistic urban environment where agents must navigate and adapt to dynamic challenges (e.g., obstacles, traffic constraints, and task allocation), in order to demonstrate the scalability and robustness of a multi-agent approach.

Operational efficiency is measured as a cost function aggregated over a simulated “day”: wage, energy, and maintenance costs plus overflow penalties. Learning agents maximize reward that is inversely related to this cost, so lower total euros correspond to higher episode rewards.

5. Restrictions

- The city is a fixed directed graph of roads and intersections which must be used by the agents. The graph represents a pre-build city in Unity.
- Agents will only collect one type of waste.
- The primary adjustable variable for performance analysis will be the number of truck agents.
- Traffic light agents will follow a fixed-time cycle and will not dynamically adjust to traffic flow.
- Each truck agent will have a maximum load capacity, after which it must return to the depot before continuing collection.
- External conditions such as rain, road works, or natural disasters will not be simulated.
- We assume that all roads are always open and in good condition.
- The central depot will remain at a fixed location and cannot be relocated during the simulation.
- Bins are placed alongside roads with a curb target.

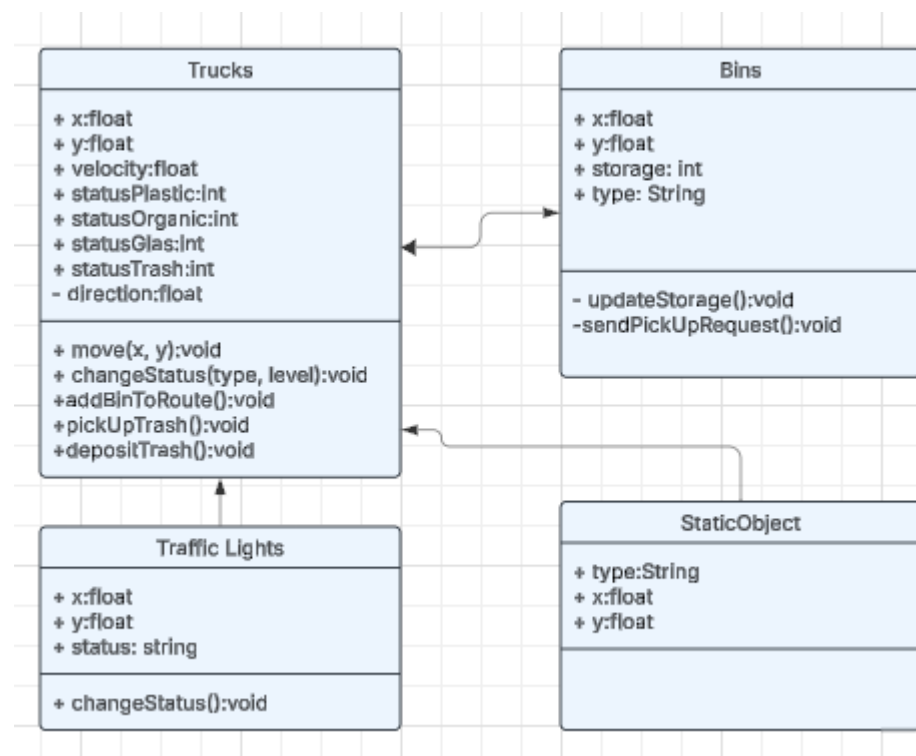
6. Requirements

- Functional:
 - FR1 (Defined by Alfredo Luce): The truck's agent must plan a route that minimizes travel time and distance, while taking into account the routing algorithm used and the amount of waste to be collected.
 - FR2 (Defined by Alfredo Luce): Truck's agent must perceive the bin fill levels, in order to collect the ones that are in yellow or red level.
 - FR3 (Defined by Marco Torres): As a system, I want to ensure that two trucks are not assigned to the same trash bin simultaneously to avoid redundancy.
 - FR4 (Defined by Fernando Hernández): The trucks must follow traffic rules, follow the correct street directions, and respect the traffic lights.
 - FR5 (Defined by Fernando Hernández): The bins should consider whether to request the trucks to collect the trash of itself at a certain capacity based on the demand on the area
 - FR6 (Defined by Cedric Schuster): The bins should reassign tasks when one truck becomes unavailable due to no rest capacity, no fuel, obstacles, to ensure the trash gets collected nonetheless.
 - FR7 (Defined by Axel Huerta): Each bin must update its color state to reflect its current status: green = empty, yellow = half, red = full.
- Non-functional:
 - NFR1 (Defined by Marco Torres): The simulation should provide a clear visual distinction between empty, partially full, and full trash bins.
 - NFR2 (Defined by Cedric Schuster): The simulation should run at a minimum of 30 frames per second (FPS) to ensure a smooth visualization and usability.

- NFR3 (Defined by Axel Huerta): UI elements, like dashboard or counters, must use clear fonts and color coding for readability.

7. Description of the multiagent system

In the current submission, trucks patrol and are dispatched via a per-tick auction rather than a heavyweight manager actor. Each bin updates its fill stochastically and emits an implicit pickup need. The auction selects truly idle trucks, avoids bins already claimed by others, and assigns a single winner per bin, eliminating dog-piling. Service occurs when a truck reaches the bin's curb. The depot visits reset load and energy. Traffic lights remain defined as a planned agent type but are not yet participating in decisions.



7.1 Agent model

The system will consist of three primary types of agents:

- Bins: These agents monitor their own fill level.
 - Beliefs: Current trash level, maximum capacity, location.
 - Plans: When the trash level exceeds a threshold, broadcast a collection request.
- Trucks: These are the mobile agents responsible for collection.
 - Beliefs: Position, heading, load, energy, planned polyline route to a target (either depot or bin), road graph, depot location, assigned bin
 - Plans: Follow the current route. When within the radius of the assigned bin, collect as much trash as capacity allows. When near full or when low energy level, re-target the dump station/depot. When reaching the

depot, dump and (optionally) recharge. If the assigned bin becomes empty, the truck clears its assignment.

- Traffic Lights: These agents control the flow of traffic at intersections.
 - Beliefs: Current state (red, yellow, green), timing cycles.
 - Plans: Cycle through states to manage traffic flow.

7.2 Environmental model

The system will be present in a complex environment, a city. For the purposes of the simulation, at least at first, only some of the characteristics of a real city will be integrated and as we continue with the development of the project we will evaluate whether or not to add more realistic features.

At first this would be the elements:

- Lanes: The “road” that the trucks will follow, according to the basics of driving in a city, making turns and using speeds coherent to the space.
- Buildings, parks, and houses: They are most part of the place, the agents will not interact with these components, given that they follow the roads and the bins will be placed close to the sidewalk, these elements will only have an aesthetic purpose.
- Traffic Lights: Part of the “rules” that the trucks will follow. The traffic lights are a real and interactive characteristic of the environment, the trucks will respect the traffic lights as any other car does in real life.
- People, benches, lights, bus stops, trees, bushes, etc.: Smaller components of the environment that give a more realistic feeling. These components, just like the buildings, will not interact with the agents.
- Cars and other vehicles: A realistic component that would interact with the trucks.

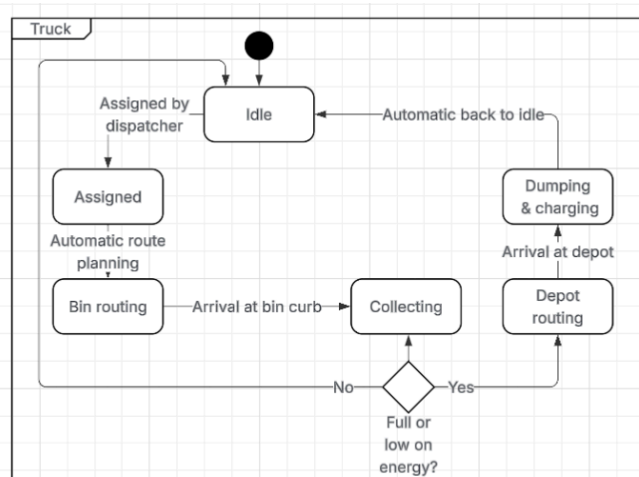
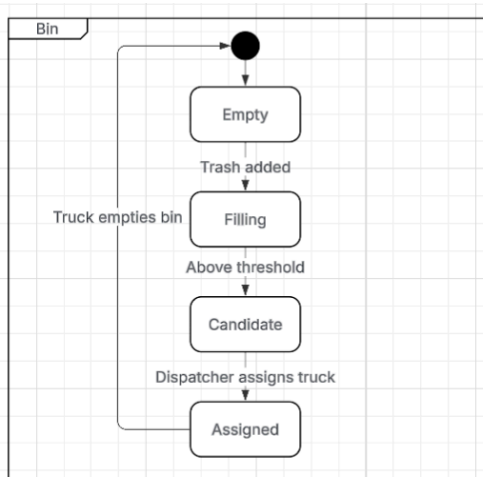
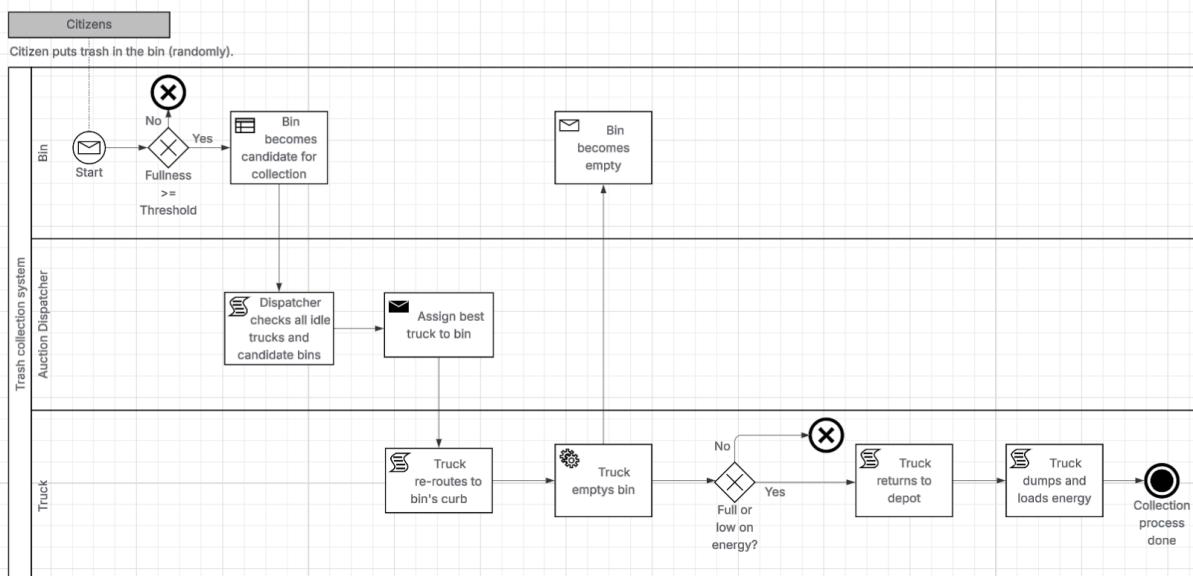
Routing is computed strictly on the waypoint graph using Dijkstra by creating a polyline route. If a direct path is not available, the planner selects the closest reachable waypoint to the goal and plans to that proxy, ensuring trucks never leave the network.

7.3 Negotiation model

This negotiation model provides the concept on how the agents (trucks) agree on who does what. This includes the assignment of a bin to a specific truck but also the re-assignment if one truck fails to collect a bin and, thus, another truck is needed. The overall idea is that a full bin requests a truck, and all available trucks propose their time of arrival. The bin accepts one proposal and therefore one truck gets the assignment.

General Procedure:

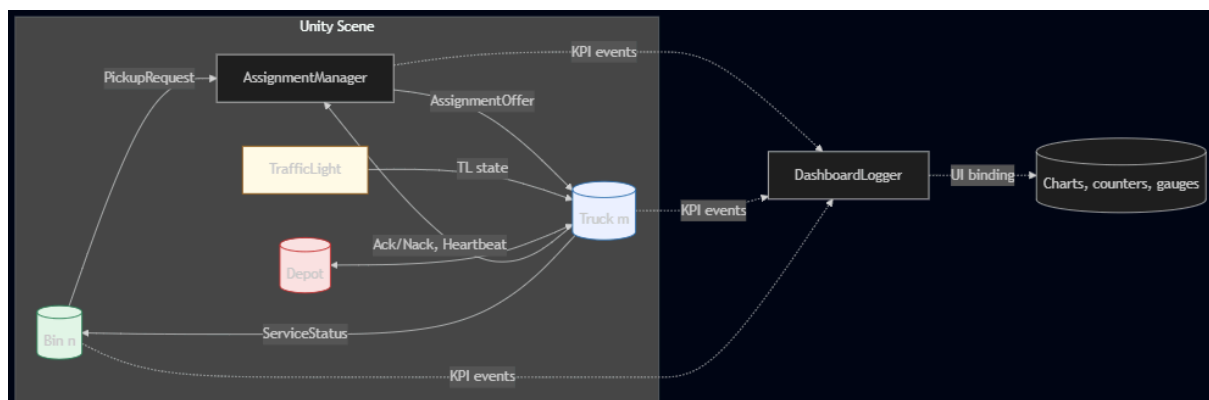
1. Idle truck check: At each tick, the system looks only at trucks that are truly idle.
2. Candidate bin selection: From all bins, it keeps only those that are non-empty and not already claimed by another truck.
3. Priority calculation: Each candidate bin is ranked: urgent bins come first, then by fill fraction.
4. Greedy matching: Urgent bins are matched first to the closest idle trucks. Then, high-fill bins and lastly, any remaining non-empty bins are assigned. Each bin is given to at most one truck.
5. Assignment & routing: The winning truck is assigned the bin and receives an immediate route plan to that bin's curb position.
6. Persistence of assignment: Once assigned, the truck keeps the bin until it is emptied or the truck fills up, preventing other trucks from converging on the same bin.
7. Reassignment on failure: If the truck becomes full earlier, runs out of energy or the bin empties before the planned pickup, the assignment is cleared and the bin becomes a candidate again at the next tick.



7.4 Interaction model

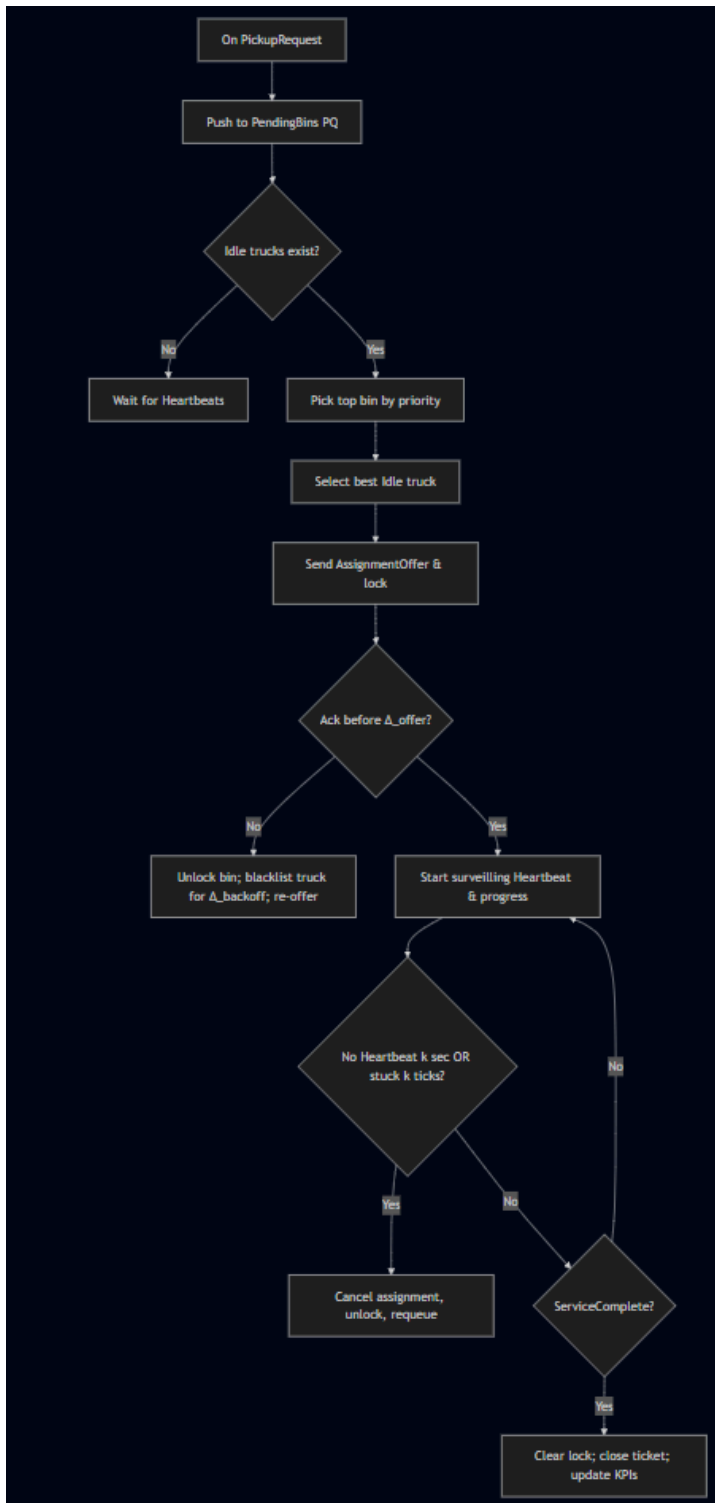
The following diagram provides an overview of the main components involved in the simulation. It shows the interactions between bins, trucks, the AssignmentManager, traffic light, the depot and the KPI logging system. This architecture outlines the key data flows and event exchanges within the system.

Agents interact through simple event messages managed by a singleton *AssignmentManager* and a *DashBoardLogger*. And no complex middleware is required.



This sequence diagram captures the full lifecycle of a bin service. It includes decision making by the AssignmentManager, traffic light interactions, the service process, and potential depot visits, along with relevant performance metrics.

The diagram below details the internal logic of the AssignmentManager when handling pickup requests. It shows how bins are prioritized, idle trucks are selected, lock policies are enforced, and failures or delays are managed. This flow ensures robustness and adaptability under dynamic conditions.



a) Message types

PickupRequest (Bin → assignmentManager): { binId, location, fillLevel, requestTime } emitted when fillLevel > T, with a short cooldown to avoid spamming.

AssignmentOffer (AssignmentManager → Truck): { binId, location priority }.

AssignmentAck/Nack(Truck $\rightarrow$ *AssignmentManager*). Accept if state == Idle, else Nack

ServiceStatus (Truck $\leftrightarrow$ Bin): { *onApproach*, *StartService*, *ServiceComplete* }.

IntersectionQuery (Truck $\rightarrow$ *TrafficLight*): request light state on approach, truck stops if red

DumpRequest / *DumpComplete* (Truck $\leftrightarrow$ *Depot*): Triggered when load > capacity or lowFuel.

Heartbeat (Truck $\rightarrow$ *AssignmentManager*): periodic current state.

b) Interaction protocol

Bin request service: process threshold T and sends *PickupRequest*.

Assignment & locking (avoids double trucks): *AssignmentManager* computes a priority for each pending bin. It sends an *AssignmentOffer* to the best Idle truck and locks that bin.

Truck commits: Truck replies *Ack* $\rightarrow$ state *NavigatingToBin*. If timeout or *Nack*, the lock is released and the offer is reissued to the next candidate.

Traffic interaction: On approaching an intersection, truck queries the *TrafficLight*. If red, it queues and waits; if green, it proceeds.

Service cycle: Truck sends *OnApproach*, then *StartService*, empties the bin (instant or short timer), updates *DashboardLogger* KPIs, and emits *ServiceComplete*. Bin resets *fillLevel* and clears its lock.

If the truck is full or low fuel, it goes to *Depot*; otherwise it requests a new assignment.

c) Local avoidance and safety

To keep a minimum following distance d_{safe} ; if another truck occupies the next node, wait 1–2 ticks.

Static obstacles: pathfinding. Recalculate if the next node is blocked > τ seconds.

Only one truck advances per lane per green tick; others queue

d) Failure and timeout handling

If no *Ack* in 2–3s, unlock the bin and re-offer.

If no *Heartbeat* for k seconds or position unchanged over k ticks, cancel assignment and requeue bin.

After *ServiceComplete*, suppress new requests for some seconds.

e) KPIs and dashboard hooks

On each key event, call *DashboardLogger.Log()* to update:

totalBinsCollected, *avgCollectionTime*, *truckIdleTime*, *truckUtilization*, *inQueueBins*.

Unity communicates with the Python server through a REST API built with FastApi.

Endpoints:

- Post /simulate: runs a simulation and returns Unity Compatible JSON.
 - If seed is null/omitted, the server creates one and echoes the run deterministically internally.
 - Streets_mask can be injected: a NxN 0/1 grid; otherwise we use a default cross-grid for corridors.
- Get /simulate: quick testing via query params.
- Get /health: server readiness probe.

JSON format (SimData):

```
{
  "grid": { "width": 150, "height": 150, "depot": [0, 0] },
  "agents": [
    {
      "id": 0,
      "start": [12, 30],
      "pathObj": [{"x":12,"y":30}, {"x":13,"y":30}, {"x":14,"y":30}],
      "distance": 48.5,
      "collected": 23,
```

```

    "capacity": 20
  }
],
"bins": [
  { "id": 5, "pos": [40, 72], "initial": 12, "remaining": 0 }
],
"events": [
  { "t": 123, "type": "ASSIGN", "agent": 0, "bin": 5, "amount": 0 },
  { "t": 178, "type": "SERVICE", "agent": 0, "bin": 5, "amount": 12 },
  { "t": 300, "type": "DUMP", "agent": 0, "bin": -1, "amount": 20 },
  { "t": 812, "type": "ALL_COLLECTED", "agent": -1, "bin": -1, "amount": 0 }
],
"metrics": {
  "total_collected": 143,
  "avg_distance_per_agent": 37.9,
  "negotiation_messages": 61,
  "steps": 1200
}
}

```

Notes:

pathObj is an ordered list of grid cells to play back in Unity.

events[].type includes ASSIGN, SERVICE, BIN_EMPTY, DUMP, ALL_COLLECTED.
agent/bin are -1 when not applicable.

metrics.steps is the final tick when run ends; avg_distance_per_agent is computed from per-agent totals.

The Unity workflow is started by the `SimulationPlater_PathObj.cs` that sends POST to `/simulate`, it receives JSON, spawns trucks and bins prefabs, then replays `PathObj` arrays to animate trucks and finally prints KPIs to console.

7.5 Deep Q-Learning

The DQN setup uses one shared policy across all trucks as the trucks are not different from each other. Each agent observes position, load, energy, its current assignment, the nearest three bins, and its id. The discrete actions are:

1. Continue moving along the current plan
2. Return to depot (sets the depot as target, then switches to move)
3. Recharge (only at depot)
4. Wait

Unsafe or nonsensical choices are masked and waiting is ignored if a commitment or route is active. Rewards are a scaled negative of incremental costs (wage, energy, maintenance), plus small positive bonuses for pickups, dumps, and efficient multi-stop tours between depots. Overflow events apply penalties.

7.6 Generation of JSON

- Producer: `server_rest.py` builds the response via `_to_unity_payload(cfg, city, sim)`. It runs the hybrid simulation (`sim.py`) and converts its state into a Unity-friendly JSON.
- Road-constrained paths: Per-truck `pathObj` is reconstructed from ASSIGN/SERVICE/DUMP events using `City.plan_route` (Dijkstra over the road graph) and expanded to 4-neighbor steps. This ensures no diagonals and no building cut-throughs. Each truck returns to the depot by the end of its path.
- Contents (top-level):
 - `grid`: { width, height, depot[x,y] } — scene bounds in grid cells and depot location.
 - `agents`: array per truck
 - `id`: integer index
 - `start`: [x,y] (depot by default)
 - `pathObj`: [{x,y}, ...] road-aligned steps (orthogonal)
 - `distance`: total path length in grid units
 - `collected`: total serviced waste amount
 - `capacity`: truck capacity (for HUD/UI)
 - `bins`: array of collection points
 - `id`: stable integer
 - `pos`: [x,y] bin position near the road curb
 - `initial`: fill at t0
 - `remaining`: final fill

- events: time-stamped operational events (see types below)
- metrics: summary { total_collected, avg_distance_per_agent, negotiation_messages, steps }
- Event types (normalized for Unity):
 - ASSIGN: truck selected to service a bin { t, type, agent, bin }
 - SERVICE: pickup from a bin { t, type, agent, bin, amount }
 - DUMP: truck emptied load at depot { t, type, agent }
 - RECHARGE: truck recharged at depot { t, type, agent }
 - OVERFLOW: a bin reached capacity { t, type, bin }
 - JSON schema: Available live at GET /schema; keeps clients in sync with any safe additions.

Tiny example (trimmed for readability): { "grid": { "width": 220, "height": 160, "depot": [150, 75] }, "agents": [{ "id": 0, "start": [150, 75], "pathObj": [{"x":150,"y":75}, ...], "distance": 1280, "collected": 540, "capacity": 300 }], "bins": [{ "id": 12, "pos": [73, 34], "initial": 14, "remaining": 100 }], "events": [{ "t": 10, "type": "ASSIGN", "agent": 0, "bin": 12 }, { "t": 22, "type": "SERVICE", "agent": 0, "bin": 12, "amount": 50 }], "metrics": { "total_collected": 1095, "avg_distance_per_agent": 320.5, "negotiation_messages": 42, "steps": 20000 } }

How paths are built:

- Targets are derived from chronological events per truck (bin curbs from ASSIGN/SERVICE and depot after DUMP).
- For each segment, City.plan_route finds a road sequence, then segments are expanded to orthogonal grid steps for Unity playback.
- Start and end are the depot; curb snapping prevents off-road departures after service.

7.7 Connection: server/API

- Transport: REST over HTTP provided by FastAPI in server_rest.py with permissive CORS (for local Unity/Play mode).
- Core endpoints:
 - GET /health → { ok: true } for liveness.
 - GET /config → current default CONFIG (read-only).
 - GET /schema → JSON schema for the Unity payload.
 - GET /simulate → one-shot simulation run; accepts query params; returns JSON payload.
 - POST /simulate → same as GET, but accepts a JSON body for overrides.
 - GET /defaults → returns server-side defaults that apply when parameters are omitted.
 - POST /defaults → set/persist server-side defaults (saved in server_defaults.json).

- Why defaults matter: Unity can always call a constant URL (e.g., GET /simulate) and you can tune behavior centrally via POST /defaults without changing the client.
- Request parameters (GET query or POST body; all optional):
 - seed, num_agents, num_waste_locations, bin_capacity, steps, planner ("graph"|"grid")
 - truck_speed (m/s), sidewalk_offset (m)
 - opportunistic_fill_frac (0..1), urgency_horizon_s
 - coverage_bias (0..1) — pushes trucks to farther bins when ties exist
 - service_cooldown_s — prevents immediate reassignment of recently serviced bins
- Response: The Unity payload described above. Content-Type: application/json. Typical 200 OK; invalid inputs return 4xx with FastAPI error details.
- Operational notes:
 - steps controls runtime and JSON size. Large values produce large payloads; tune for your scene.
 - planner=graph uses the road graph; planner=grid uses a grid A* fallback (optional).
 - Defaults persist across runs; override per request via query/body.

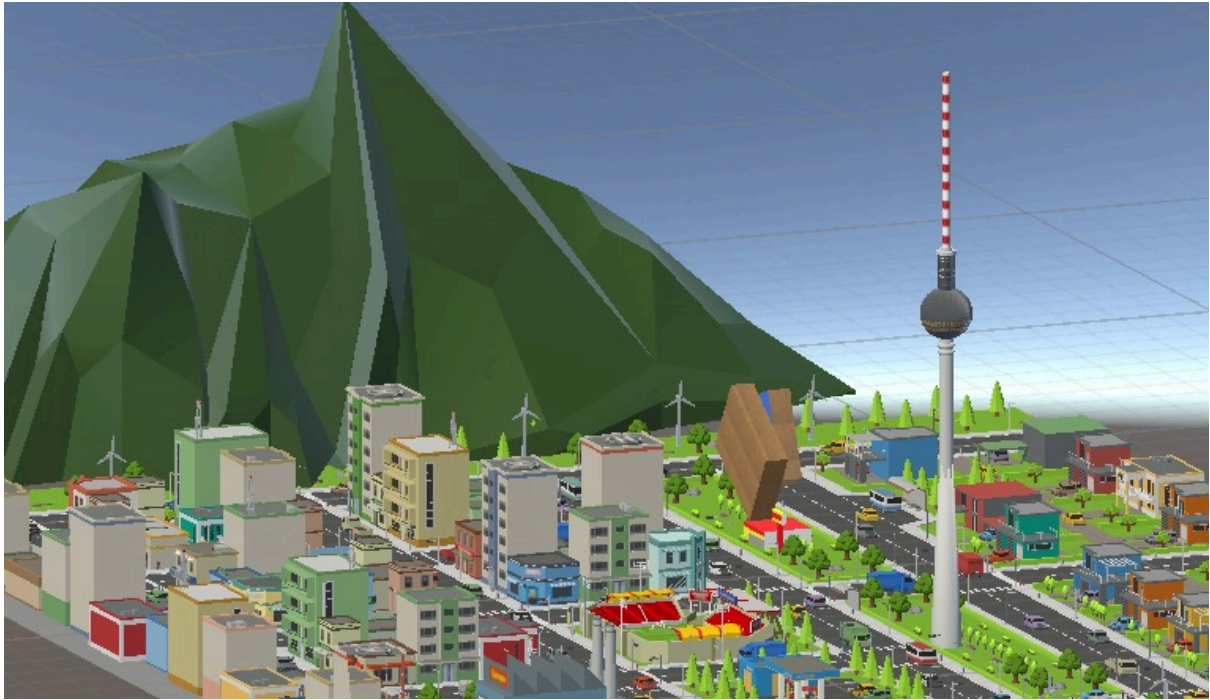
7.8 Unity client integration

- The SimulationPlayer_PathObj.cs demonstrates:
 - Fetching JSON via GET/POST /simulate (single URL).
 - Mapping world coordinates using grid.width/height, worldOrigin, and cellSize.
 - Lane offsets to reduce visual overlap; optional local path reconstruction from events if desired.

8. Description of the graphic modeling

8.1 Scene to model

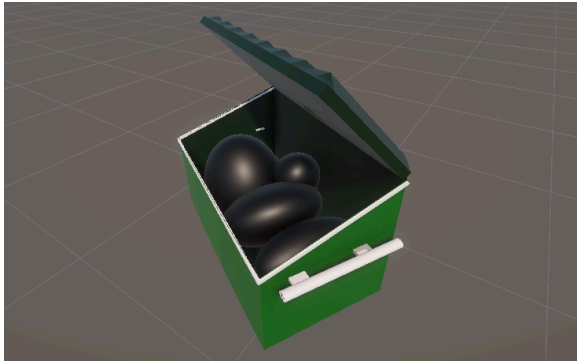
Expectation: A grid-based city layout with distinct residential and industrial zones. Roads will have intersections with traffic lights. Buildings, trees, and other static objects will be populated the scene to create a visually representative urban environment.



The reference image shows a low-poly city with a grid layout, green spaces, and different building types. We will aim to create a similar aesthetic, with apartment buildings, factories, and houses. The scene will include details like streetlights, crosswalks, parked cars, and pedestrians to enhance realism. The static elements will not interact with the agents, except for the lanes where the truck will be moving and the bins will be placed. The traffic lights have a color and can change color to green, yellow and red. The truck can move in every direction (as long as its movement is realistic and it's done inside the streets), collect garbage, unload garbage, measure how full it is and communicate with the bins and other trucks. The bins can measure how full they are and communicate with the trucks. We have added to the scene our own assets such as the buildings and the cerro de la silla.

8.2 Graphic components

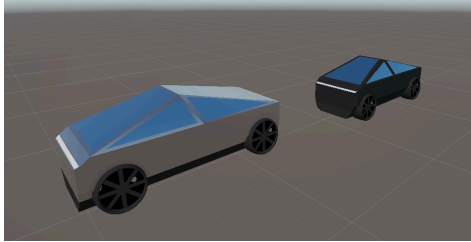
Name	Trash container
Description	A trash container type for any type of trash

Image	
Source	Alfredo Luce Morales

Name	Generic car
Description	A car based of a poly model vehicle
Image	
Source	Alfredo Luce Morales

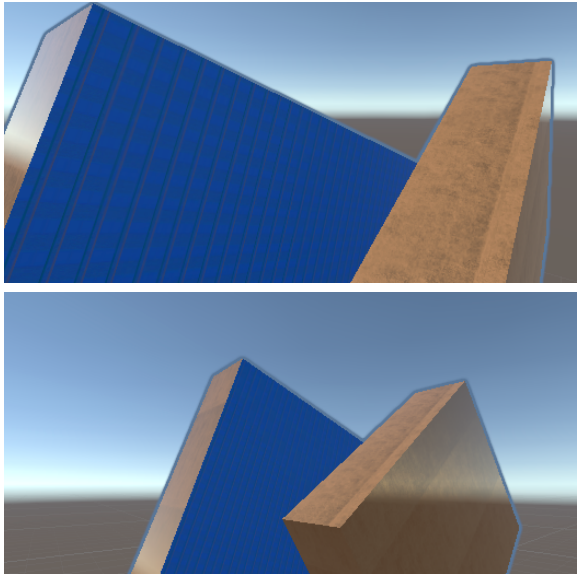
Name	Berlin TV Tower
Description	Monument, building, tourist attraction in Berlin.
Image	

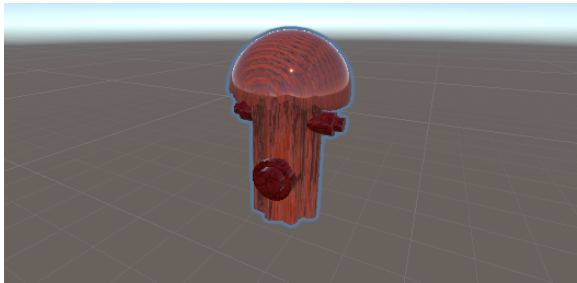
Source	Cedric Schuster
---------------	-----------------

Name	Cyber Truck
Description	Car model by Tesla, in two different colors (shiny metal and matte black).
Image	
Source	Cedric Schuster

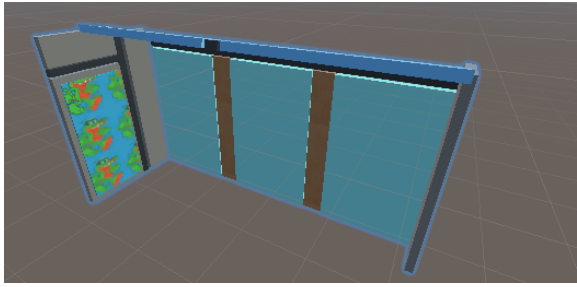
Name	SUPRA TOYORA
Description	A sports car of the toyota brand
Image	
Source	Fernando Hernández

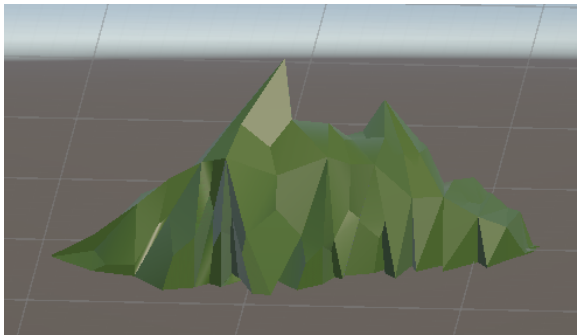
Name	Servilletero
Description	Building in the Tecnológico de Monterrey

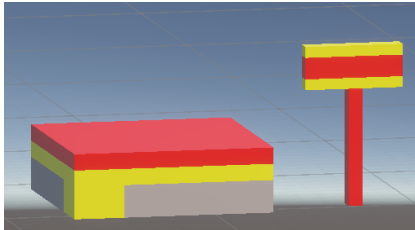
Image	
Source	Fernando Hernández

Name	Hydrant
Description	A hydrant that can be found at corners, used by fire fighters to connect to the water network in the city.
Image	
Source	Marco Torres

Name	Bus Stop
Description	A place where people can sit to wait for the bus to arrive, it can be found all around the city aside the road.

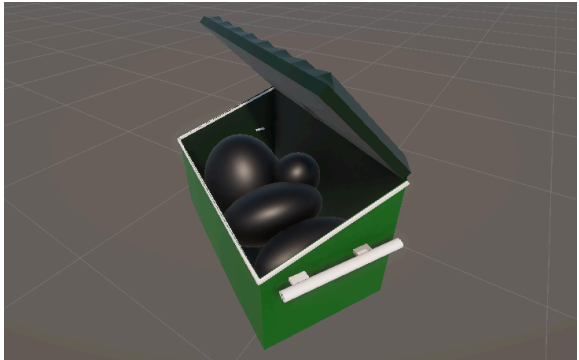
Image	
Source	Marco Torres

Name	Cerro de la silla
Description	Protected natural monument icon of Monterrey, Mexico, recognized by its saddle-like shape
Image	
Source	Axel Huerta

Name	Oxxo
Description	Convenience store chain, offering everyday products, prepaid services, etc.
Image	
Source	Axel Huerta

8.3 Prefabs

Name	Truck
Description	Garbage truck
Image	
Source	Low Poly Assets

Name	Trash container
Description	A trash container type for any type of trash
Image	
Source	Alfredo Luce Morales

8.4 Scripts

Name	dqn_agent.py
-------------	--------------

Description	Advanced DQN (Double, Dueling, optional Prioritized Replay) with experience buffer, epsilon schedule, soft target updates.
--------------------	--

Interactions	Used by rl_policy.DQNManager; hyperparameters from config; affects truck decisions via Simulation when POLICY='dqn'.
---------------------	--

Name	train_dqn.py
-------------	--------------

Description	Stand-alone training loop with evaluation, logging, checkpoints, early stopping.
--------------------	--

Interactions	Uses dqn_agent, simulation components, reads config, produces weights for rl_policy.
---------------------	--

Name	server_rest.py
-------------	----------------

Description	FastAPI server exposing simulation, defaults, async jobs, preview, and Unity payload builder.
--------------------	---

Interactions	Interactions: Imports config, city, sim, dispatch, ap_model; provides data to Unity/clients; uses Simulation events.
---------------------	--

Name	config.py
-------------	-----------

Description	Central parameters for map, roads, depot, bins, trucks, energy, RL, dispatch, lanes.
--------------------	--

Interactions	Imported by most modules (city, sim, rl_policy, dqn_agent, server_rest); drives all behavior.
---------------------	---

Name	city.py
Description	Road/waypoint graph (and optional grid) with route planning and bin placement.
Interactions	Called by Simulation and dispatcher for plan_route; used by exporter for waypoint indexing.

Name	agents.py
Description	BinObj fill/overflow logic and Truck movement, routing, service, lane offset, energy/cost updates.
Interactions	Instantiated in Simulation; manipulated by dispatch and rl_policy; events consumed by exporter.

Name	dispatch.py
Description	Auction-based assignment with urgency, coverage bias, anti-tailgating.
Interactions	Called each step by Simulation when POLICY='auction'; sets truck routes; emits assign/tailgate events.

Name	rl_policy.py
-------------	--------------

Description DQNManager bridging environment state to DQN actions (candidate generation, reward handling).

Interactions Uses dqn_agent; invoked by Simulation when POLICY='dqn'; assigns trucks similar to dispatch.

Name sim.py

Description Core simulation loop handling bin fill, assignments, movement, events, RL learning, frame logging.

Interactions Used by server_rest, training script; uses agents, dispatch or rl_policy, city for routes.

Name grid_planner.py

Description A* and Manhattan fallback for grid-based pathfinding.

Interactions Used in Simulation._plan_route when planner='grid'.

Name export_unity.py

Description Stand-alone exporter building Unity-friendly paths/events outside the server.

Interactions Consumes Simulation + City; mirrors logic in server_rest payload generation.

Name	ap_model.py
Description	AgentPy integration wrapper enabling alternate model execution path.
Interactions	Optionally used by server_rest to run via agentpy; still feeds exporter functions.

Name	viz_rich.py
Description	Rich terminal visualization/debug helpers for simulation state.
Interactions	Optional developer tool reading Simulation objects and config.

GitHub Repository Link: <https://github.com/FernandGS/MA-Systems-Challenge>

9. Project management deliverables

<https://dev.azure.com/GridAndGrit/Trash%20Collection%20System>

10. References

- Alfeo, A. L., Ferrer, E. C., Lizarribar Carrillo, Y., Grignard, A., Pastor, L. A., Sleeper, D. T., Cimino, M. G. C. A., Lepri, B., Vaglini, G., Larson, K., Dorigo, M., & Pentland, A. (2018). Urban swarms: A new approach for autonomous waste management. <https://www.media.mit.edu/publications/urban-swarms-a-new-approach-for-autonomous-waste-management/>
- Bharadhwaj, S. P., Sampath, A., Vishwas, A. M., Chandra, S. R., & Krishnan, R. (2025, April). Dynamic route optimization for urban waste collection. *International Journal for Research in Applied Science & Engineering Technology (IJRASET)*.

<https://www.ijraset.com/best-journal/dyanamic-route-optimisation-for-urban-waste-collection>

Hess, C., Dragomir, A.G., Doerner, K.F. et al. Waste collection routing: a survey on problems and methods. *Cent Eur J Oper Res* 32, 399–434 (2024). <https://doi.org/10.1007/s10100-023-00892-y>

Gautam, S., & Velijala, V. (2024, September 23). Sustainable organic waste management: A playbook for Lucknow, India. <https://www.teriin.org/policy-brief/sustainable-organic-waste-management-playbook-lucknow-india>

Karadimas, N. V., et al. (n.d.). Coupling multi-agent simulation and GIS: Towards intelligent decision support in waste management. <https://citeseerx.ist.psu.edu/document?doi=7858a054203584fd9f3306359dc1dbd8d623db74&repid=rep1&type=pdf>

Likotiko, E., Nyambo, D., & Mwangoka, J. W. (2017). Multi-agent based IoT smart waste monitoring and collection architecture. https://www.researchgate.net/publication/228747937_Coupling_Multiagent_Simulation_and_GIS_-_an_Application_in_Waste_Management

Pires, L. M., Figueiredo, J., Martins, R., & Martins, J. (2025). IoT-Enabled Real-Time Monitoring of Urban Garbage Levels Using Time-of-Flight Sensing Technology. *Sensors*, 25(7), 2152. <https://doi.org/10.3390/s25072152>

Smart waste for factories, facility managers and Retail. Sensoneo. (2023, July 28). <https://sensoneo.com/es/residuos-colectores/>

VenCreations. (n.d.). Simplepoly City - Low poly assets: 3D Environments: Unity Asset Store. 3D Environments | Unity Asset Store. <https://assetstore.unity.com/packages/3d/environments/simplepoly-city-low-poly-assets-58899>

Wouda, N. A., Aerts-Veenstra, M., & van Foreest, N. (2024, September). An integrated selection and routing policy for urban waste collection. <https://arxiv.org/abs/2409.13386>